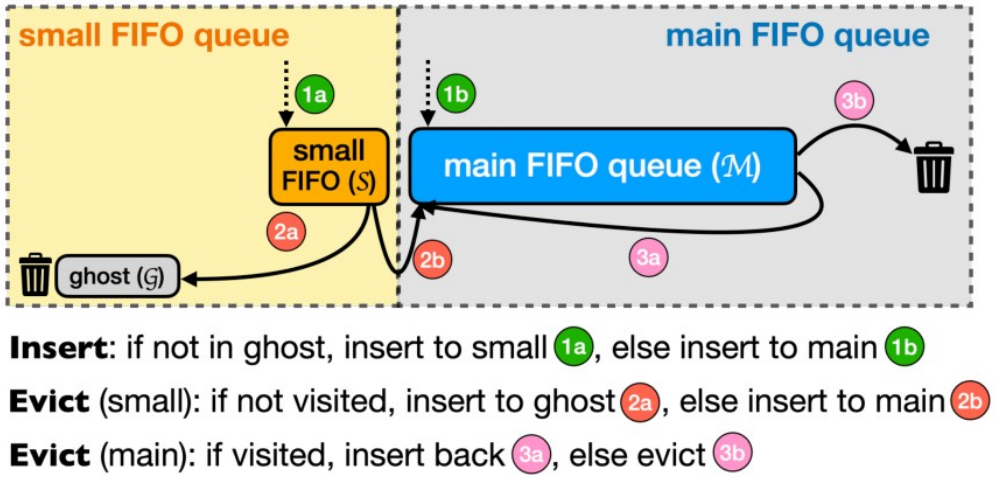
OPERATING SYSTEMS · PAPER PRESENTATION

S3-FIFO: Cache Eviction

FIFO Queues Are All You Need for Cache Eviction



Original Figure 5 · S3-FIFO architecture

Source: Yang et al., SOSP 2023

Three members · 10 minutes

SOSP 2023 · Juncheng Yang et al.

Team Structure & Timing

Three speakers follow the supplied script: background, algorithm, then evaluation.

Speaker	Time	Main responsibility
Member 1	3 min	Background, cache eviction, FIFO / LRU, one-hit wonders.
Member 2	4 min	Three FIFO queues, cache requests and eviction rules.
Member 3	3 min	Experimental setup, results, limitations and conclusion.

10-minute theoretical presentation · Results are reported by the original paper's authors.

PART 1 · MEMBER 1

Background & Motivation

Why does a cache need a better eviction policy?

Background → FIFO vs LRU → One-hit Wonders

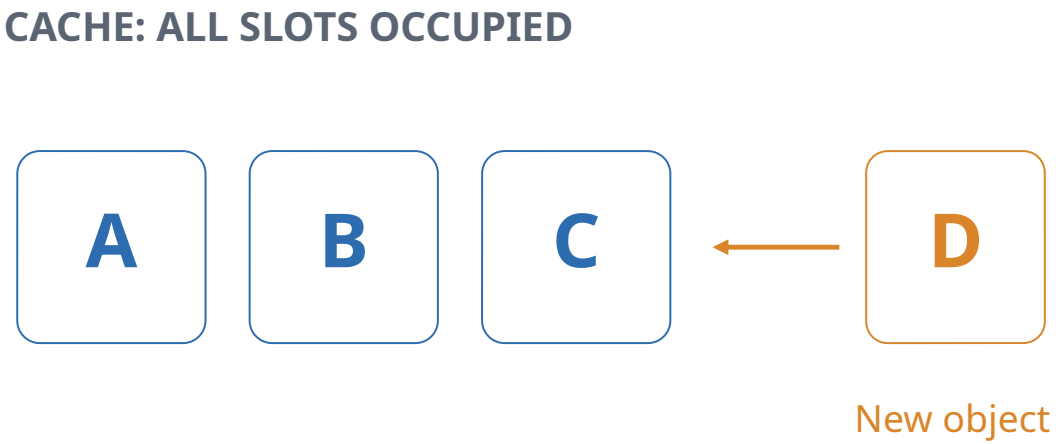
What Is Cache Eviction?

Cache space is fast but limited; the system must decide what stays and what goes.

A cache is a small and fast storage space that keeps data we may need again.

When the cache is full, the system must decide which data to remove.

This process is called cache eviction.



Key question: what should stay in the cache, and what should go?

Source: Yang et al., SOSP 2023, §§1–2.

FIFO vs LRU

Both choose eviction candidates, but their update costs are different.

FIFO

First In, First Out

It removes the data that entered the cache first.

FIFO is simple and fast, but it may remove useful data.

LRU

Least Recently Used

It removes data that has not been used for the longest time.

LRU updates object order on access. This can slow the system down when many threads work together.

The goal: combine FIFO's simplicity with better cache efficiency.

Source: Yang et al., SOSP 2023, §2.2.

One-Hit Wonders & Quick Demotion

Shorter access sequences contain a higher fraction of objects with no reuse.

Many objects are used only once before they leave the cache: one-hit wonders.

They take up cache space without being used again.

Quick demotion removes less useful objects early.

A SHORTER SEQUENCE → A HIGHER RATIO



start time	end time	sequence length (# objects)	# one-hit wonder	one-hit-wonder ratio
1	17	5	1 (E)	20%
1	7	4	2 (C, D)	50%
1	4	3	2 (B, C)	67%

Original Figure 1 · Source: Yang et al., SOSP 2023

Can we combine the simplicity of FIFO with better cache efficiency?

Source: original Figure 1 and §3, Yang et al., SOSP 2023.

PART 2 · MEMBER 2

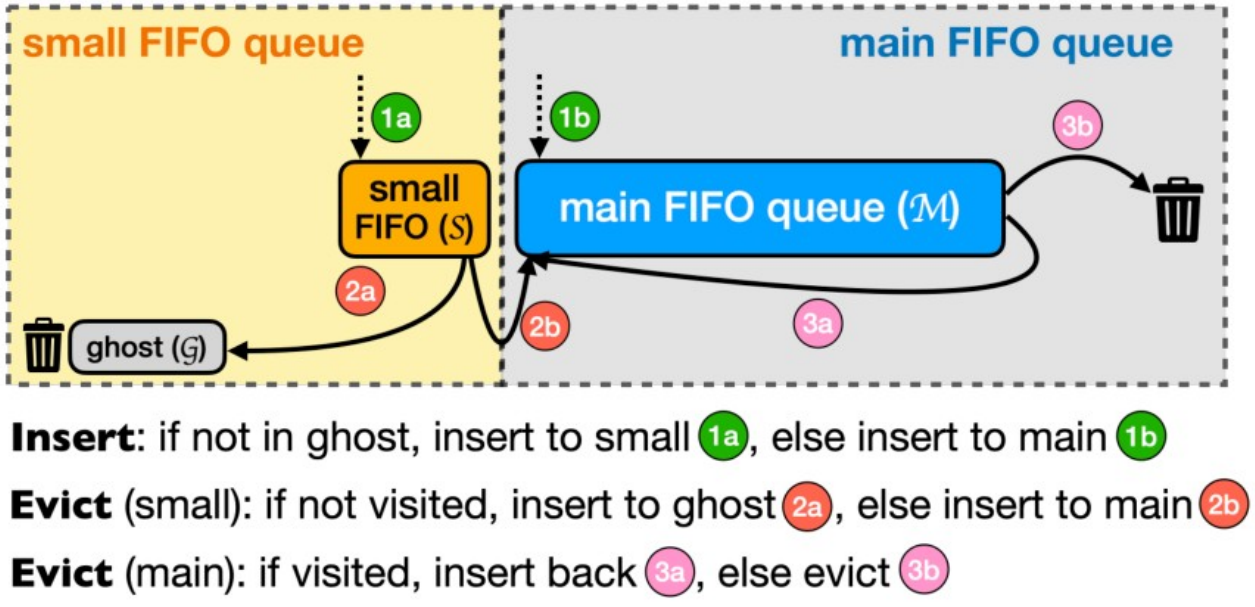
S3-FIFO Design & Algorithm

How do three simple FIFO queues work together?

Small / Main / Ghost → Requests → Eviction

Three FIFO Queues, One Cache

S3-FIFO uses Small, Main and Ghost queues instead of a single cache queue.



Original Figure 5 · Source: Yang et al., SOSP 2023

S **Small**

Filter new objects

M **Main**

Keep useful objects

G **Ghost**

Remember removed IDs

The main idea: use three FIFO queues to filter, protect and remember.

Source: Figure 5 and §4.1, Yang et al., SOSP 2023.

Small, Main and Ghost

Small and Main store data; Ghost keeps only metadata for recently removed objects.



SMALL QUEUE

10%

New objects usually enter S first.

A testing area: objects with no reuse leave quickly.

MAIN QUEUE

90%

M keeps objects that have shown they are useful.

Useful objects stay in cache for longer.

GHOST QUEUE

IDs only

G remembers recently removed object IDs.

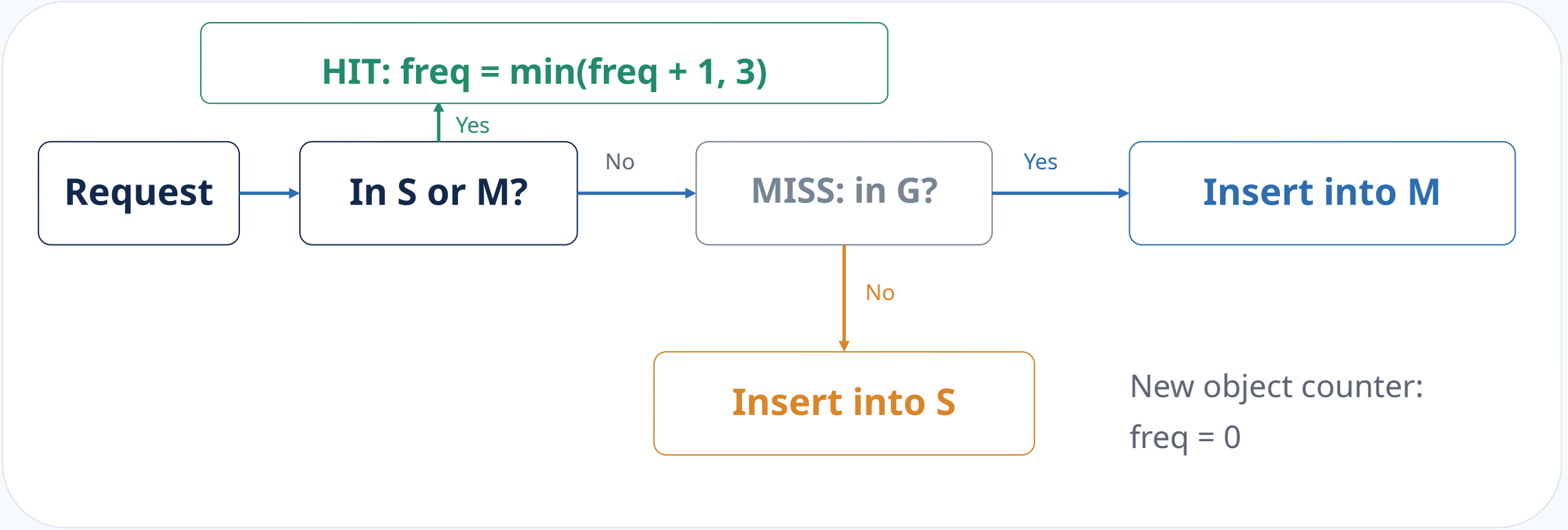
It stores no actual cached data.

Ghost entries are metadata; they are not part of the cached data payload.

Source: §4.1, Yang et al., SOSP 2023.

Cache Hit and Cache Miss

A hit updates a capped access counter; a miss uses Ghost history to choose a queue.



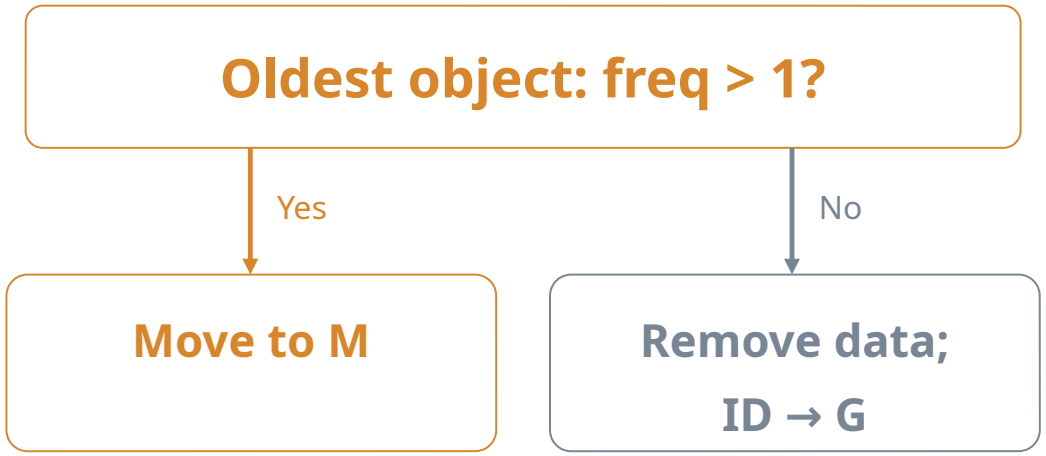
Unlike LRU, S3-FIFO does not reorder objects after every cache hit.

Source: Algorithm 1, lines 1–13, Yang et al., SOSP 2023. Simplified editable flowchart.

Eviction: Filter or Give Another Chance

When space is needed, S filters low-reuse objects while M protects accessed objects.

SMALL QUEUE / S



Objects with enough reuse enter Main.

MAIN QUEUE / M



An accessed object gets another chance.

Quick demotion + selective reinsertion → fewer queue-order updates on cache hits.

Source: Algorithm 1, lines 14–40, Yang et al., SOSP 2023. Simplified editable flowcharts.

PART 3 · MEMBER 3

Evaluation & Conclusion

What do the paper’s experiments show?

Experimental setup → Efficiency → Scalability

Experimental Setup

The researchers compared S3-FIFO with FIFO, LRU and other cache replacement methods.

6,594
real cache traces

14
datasets / data groups

The data covered storage blocks, key-value data and web content.

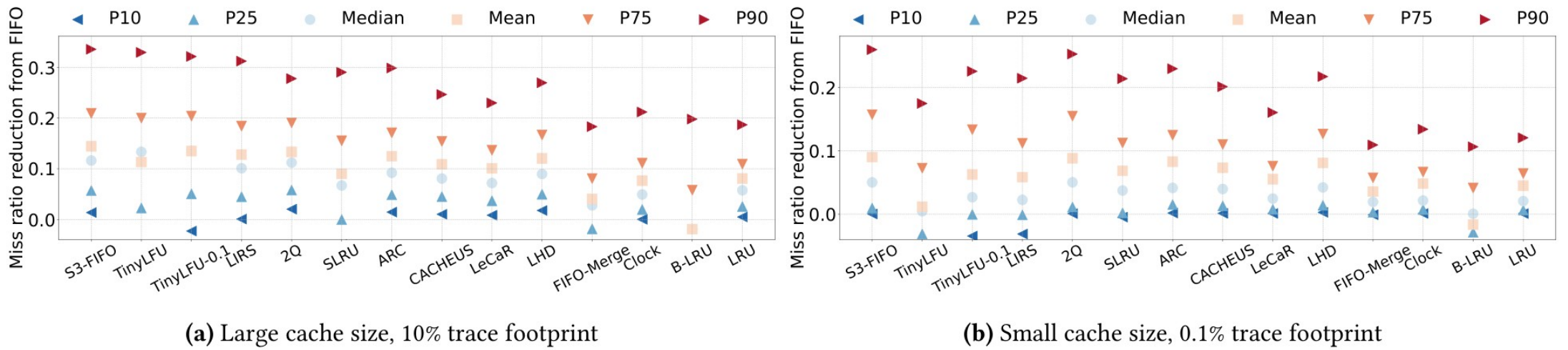
The two key measures are miss ratio and throughput.

All experimental results are reported by the original paper's authors.

Source: Table 1 and §5.1, Yang et al., SOSP 2023. This is a theoretical presentation, not a replication.

Result 1: Cache Efficiency

A lower miss ratio means more requests can be answered directly from the cache.



(a) Large cache size, 10% trace footprint

(b) Small cache size, 0.1% trace footprint

Original Figure 6 · Miss-ratio reduction relative to FIFO: a larger reduction is better.

10 / 14

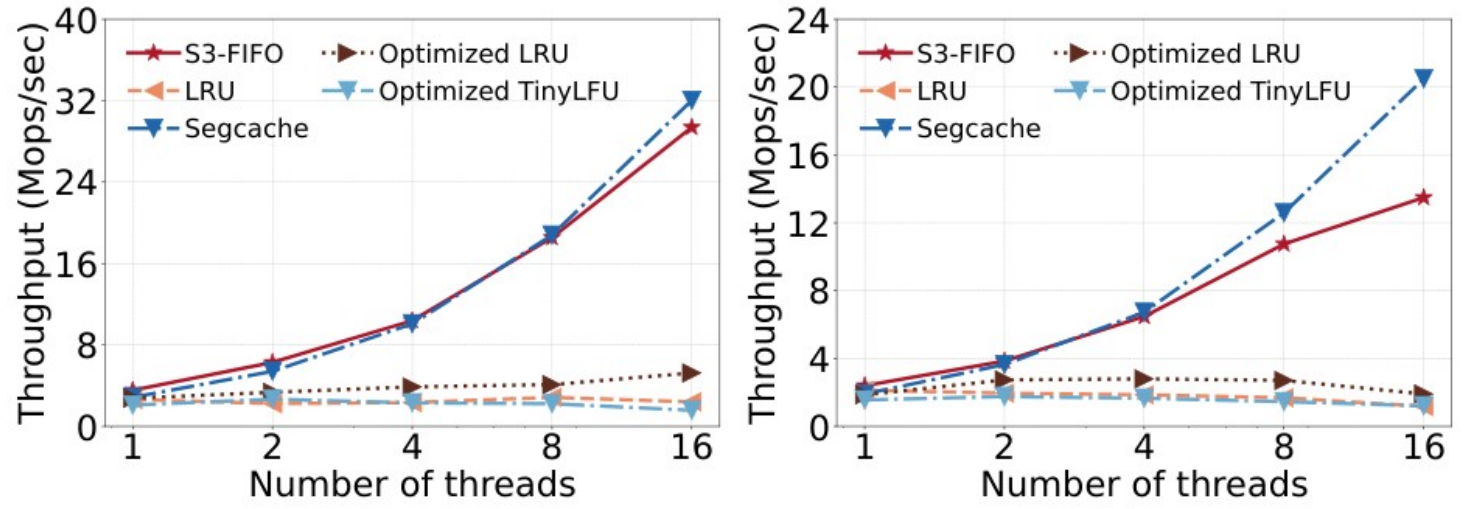
Lowest average miss ratio in 10 of 14 datasets with a relatively large cache.

S3-FIFO performs well across many workloads.

Source: Figure 6 and §5.2, Yang et al., SOSP 2023. The 10/14 dataset result is reported in §5.2 / Figure 7.

Result 2: Throughput & Scalability

Throughput means how many requests a system can handle in one second.



(a) Large cache, LRU miss ratio 0.02 **(b)** Small cache, LRU miss ratio 0.21

Original Figure 8 · Synthetic Zipf workload (α = 1.0)

>6×

Compared with optimized LRU at 16 threads.

S3-FIFO scales as the number of threads grows.

Fewer operations on cache hits support better performance under concurrency.

Source: original Figure 8 and §5.3, Yang et al., SOSP 2023. Reported authors' experiments.

A Limitation: The Second Request May Arrive Late

Performance still depends on how users access the data.

Sometimes an object is removed from the Small Queue before its second request arrives.

This can cause an extra cache miss.



Quick demotion is useful, but a late reuse can still cause an additional miss.

Source: §5.2, Yang et al., SOSP 2023.

Final Takeaway

A simple cache algorithm can still deliver strong performance.

Three FIFO queues remove less useful data quickly while keeping useful data for longer.

Lower miss ratios · Better scalability

Thank you for listening.

References

Primary paper, original figures, and the official code and data.

PRIMARY PAPER

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023).

FIFO Queues Are All You Need for Cache Eviction. SOSP ’23.

DOI

https://doi.org/10.1145/3600006.3613147

OFFICIAL CODE & DATA

https://github.com/TheSys-lab/sosp23-s3fifo

Figures 1, 5, 6 and 8: Yang et al., SOSP 2023. Mechanism diagrams redrawn from Algorithm 1.